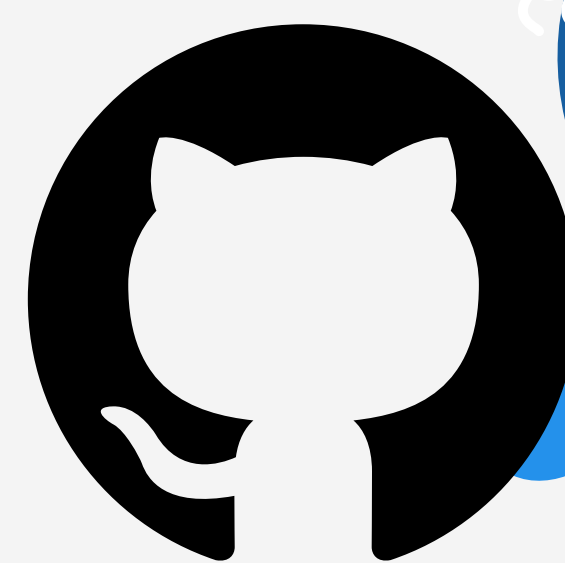




MÓDULO I



DESARROLLO WEB



¿ QUÉ ES EL DESARROLLO WEB?

El desarrollo web es la creación de sitios y apps para internet, combinando el frontend (lo que ves y usas en la pantalla) con el backend (el motor oculto en los servidores que procesa datos y bases de datos).



¿QUÉ ES HTML?

- HyperText Markup Language — lenguaje de marcado para crear páginas web
- Define la estructura y contenido de un sitio
- Usa etiquetas para organizar cada elemento de la página



¿PARA QUÉ SIRVE HTML?

- Crear la estructura base de una página web
- Organizar textos, imágenes, enlaces y formularios
- Permitir que los navegadores entiendan el contenido



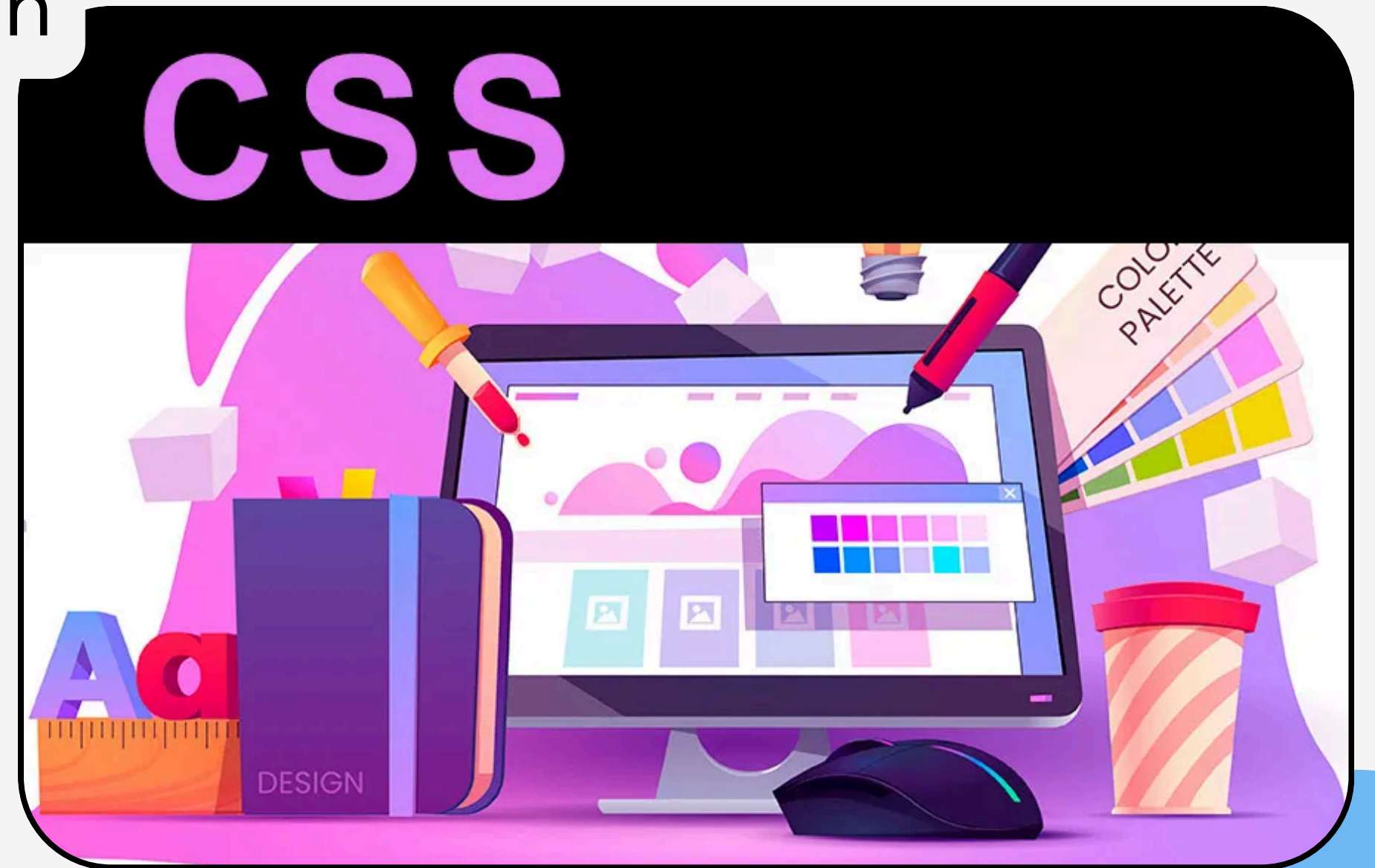
¿QUÉ ES CSS?

- Cascading Style Sheets –
Lenguaje para dar estilo y
diseño a las páginas web
- Control visual: Colores, tamaños,
espacios y posiciones de cada
elemento
- Separación: Mantiene el diseño
separado del contenido HTML



¿PARA QUÉ SIRVE CSS?

- Hacer que las páginas se vean atractivas
- Crear diseños responsivos (adaptables a móviles)
- Separar el contenido del diseño visual



¿QUÉ ES GIT?

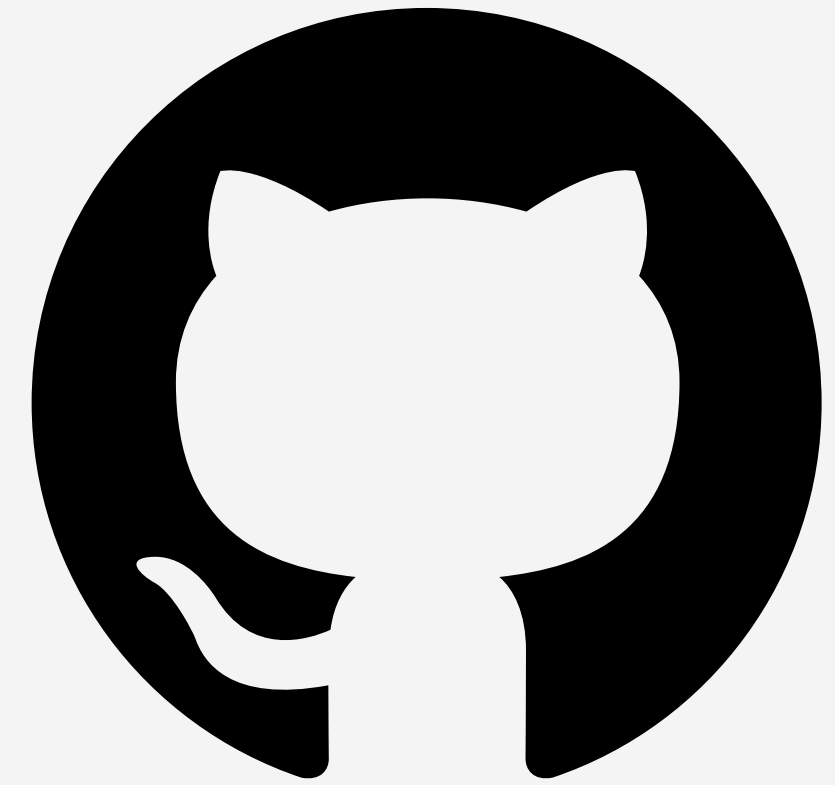
- Sistema de control de versiones para código.
- Registra todos los cambios en tus proyectos.
- Permite trabajar en equipo sin conflictos.



Git guarda un historial completo de tu proyecto puedes volver a cualquier versión anterior en cualquier momento.

¿QUÉ ES GITHUB?

- Almacena repositorios Guarda tus proyectos Git en la nube de forma segura.
- Colaboración Facilita el trabajo en equipo entre desarrolladores.
- Portafolio Comparte y muestra tus proyectos al mundo.



HTML: FUNDAMENTOS

Conoce la estructura básica que toda página web necesita para existir.



ESTRUCTURA BÁSICA DE HTML



<!DOCTYPE html> – declara que es HTML5

<html> – Contenedor principal de la página

<head> – Información y metadatos de la página

<body> – Todo el contenido visible

Estructura básica de un documento HTML

```
<!DOCTYPE html>
<html>
  <head>
    Información que lee el navegador
  </head>
  <body>
    Contenido que ve el usuario
  </body>
</html>
```

EJEMPLO DE HTML

index.html X

index.html > html

```
1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <title>Document</title>
5      </head>
6      <body>
7          <h1>Hola, Mundo!</h1>
8      </body>
9  </html>
```

<!DOCTYPE html> – Indica al navegador la versión de HTML

<title> – Aparece en la pestaña del navegador.

<h1> – El primer texto visible de la página



ALGUNAS ETIQUETAS DENTRO DEL `<BODY>`



- `<h1>` a `<h6>`: Encabezados de text
- `<p>`: Define párrafos de texto estándar.
- `<a>`: Enlaces o hipervínculos para navegar a otras páginas o secciones (`href`).
- `<img>`: Inserta imágenes especificando su ruta (`src`).
- `<div>`: Contenedor genérico en bloque para agrupar elementos y aplicar estilos.



ALGUNAS ETIQUETAS DENTRO DEL `<BODY>`



- `<span>`: Contenedor en línea para aplicar estilos a fragmentos específicos de texto.
- `<ul>`, `<ol>`, `<li>`: Listas desordenadas con viñetas, ordenadas con números, y sus elementos individuales.
- `<form>`, `<input>`, `<button>`: Elementos interactivos para recopilar datos del usuario (formularios, campos de texto y botones).
- `<header>`, `<nav>`, `<main>`, `<footer>`: Etiquetas semánticas para estructurar la cabecera, navegación, contenido principal y pie de página.

¿CÓMO SE APLICA CSS?



Existen tres formas principales de integrar CSS en tus documentos HTML, cada una con sus ventajas.

1

Inline

Estilos aplicados directamente a un elemento HTML usando el atributo `style`. Ideal para cambios rápidos y específicos, pero no recomendado para mantener un diseño consistente.

2

Interno

Estilos definidos dentro de la etiqueta `<style>` en la sección `<head>` del documento HTML. Útil para páginas individuales con estilos únicos, mantiene el CSS dentro del archivo HTML.

3

Externo

Estilos almacenados en un archivo `.css` separado y enlazados al HTML. La práctica más recomendada para proyectos grandes, permite reutilizar estilos y facilitar el mantenimiento.



SINTAXIS BÁSICA DE CSS



Cada regla CSS define cómo se verá un elemento HTML, siguiendo una estructura simple de selector, propiedad y valor.

```
selector {  
  propiedad: valor;  
  propiedad2: valor2;  
}
```

Selector: Define a qué elemento HTML se aplicará el estilo (ej. h1, .clase).

Propiedad: La característica visual que se desea modificar (ej. color, font-size, background-color).

Valor: El ajuste específico para la propiedad (ej. blue, 16px)



SELECTORES CSS



Los selectores son patrones utilizados para seleccionar y dar **estilo** a elementos específicos en tu documento **HTML**.

Selector de Elemento

Afecta a todos los elementos de un tipo específico, como **<p>** párrafos o **<h1>** para títulos

Selector de Clase

Selecciona elementos que comparten un atributo **class** específico. Se define con un punto: **.mi-clase**

Selector de ID

Selecciona un único elemento con un atributo **id** específico. Se define con un numeral: **#mi-id**

ACTIVIDAD #1



CONSTRUYE UN PEQUEÑO SITIO WEB (BLOG, WIKI, GALERIA DE FOTOS) CON LA IA DE TU PREFERENCIA UTILIZANDO HTML Y CSS. LOS ESTILOS DEBEN SER EXTERNOS ES DECIR, UN ARCHIVO DE HTML Y UN ARCHIVO DE CSS.

CONFIGURAR GIT Y GITHUB



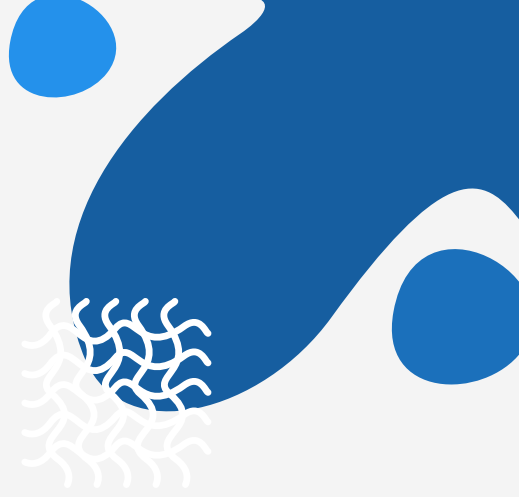
Git necesita saber quién es el que está escribiendo el código para firmar cada cambio (commit). Ejecuta estos dos comandos en tu terminal (cambiando tus datos): (en CMD, PowerShell o Git Bash)

escribe:

```
git config --global user.name "Tu Nombre"
```

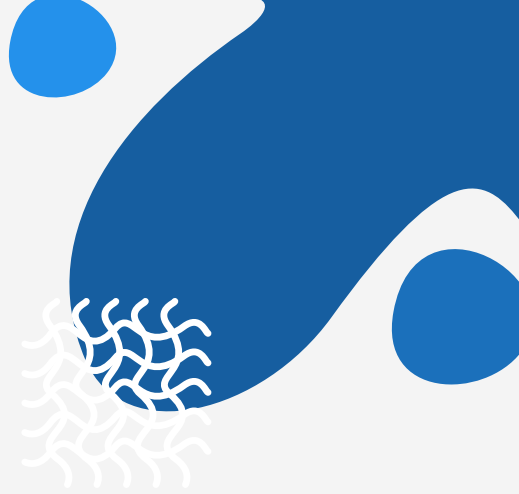
```
git config --global user.email"correo@example.com"
```

CONFIGURAR GIT Y GITHUB



1. Inicializa Git en la carpeta: `git init`
2. Crea un archivo de prueba (por ejemplo, puedes crear un archivo llamado `index.html` con cualquier texto adentro, o simplemente crearlo desde tu editor/explorador).
3. Revisa el estado para ver que Git detectó el archivo nuevo: `git status`
4. Manda el archivo a la "sala de espera" (Staging Area): `git add .`
5. Toma la foto oficial (Commit) con un mensaje descriptivo: `git commit -m "Primer commit: mi archivo inicial de prueba"` Una vez que hagas esto, ya tendrás tu historial guardado localmente.

CONFIGURAR GIT Y GITHUB



6. Entra a github.com e inicia sesión.

7. Haz clic en el botón + (arriba a la derecha) y selecciona "New repository".

8. Ponle el mismo nombre que tiene tu carpeta local (o el que quieras).

9. ¡IMPORTANTE! Asegúrate de que el repositorio esté configurado como "Public" (o "Private", como prefieras) y NO marques ninguna de las casillas (no crees el README, ni el .gitignore, ni la licencia). Queremos que el repositorio esté totalmente vacío al principio.

10. Haz clic en "Create repository".

CONFIGURAR GIT Y GITHUB



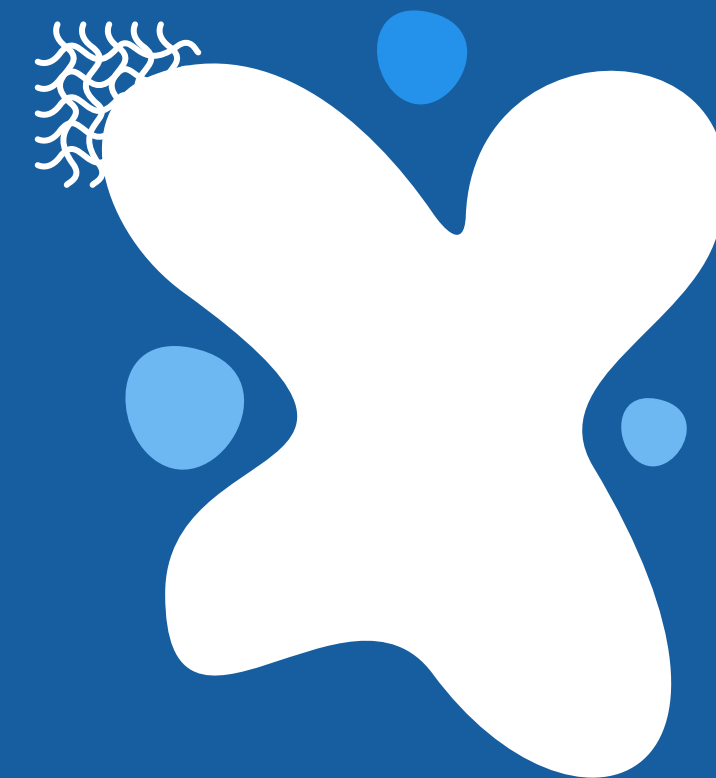
11. GitHub te mostrará una pantalla con comandos. Copia y pega estos tres en tu terminal, uno por uno:

12. Vincula tu carpeta local con el repositorio de la nube:(Copia la URL que aparece en la pantalla de GitHub de tu nuevo repositorio): `git remote add origin https://github.com/TU_USUARIO/NOMBRE_DEL_REPO.git`

13. Renombra tu rama local a main (para que coincida con el estándar de GitHub): `git branch -M main`

14. Sube tus archivos a la nube: `git push -u origin main`

Posterior a eso todo tu proyecto estará alojado en la nube con github



¡GRACIAS!

